RISC-V Assembly Quick Reference Card

RISC-V Instruction Set

Core Instruction Formats

31 – 25	24 – 20	19 – 15	14 – 12	11 – 7	6 – 0	
funct7	rs2	rs1	funct3	rd	opcode	R-type
imm[11:0]		rs1	funct3	rd	opcode	I-type
imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode	S-type
imm[12|10:5]	rs2	rs1	funct3	imm[4:1|11]	opcode	B-type
imm[31:12]				rd	opcode	U-type
imm[20|10:1|11|19:12]				rd	opcode	J-type

Registers

Register	ABI Name	Description	Saver
x0	zero	Zero constant	—
x1	ra	Return address	Callee
x2	sp	Stack pointer	Callee
x3	gp	Global pointer	—
x4	tp	Thread pointer	—
x5-x7	t0-t2	Temporaries	Caller
x8	s0 / fp	Saved / frame pointer	Callee
x9	s1	Saved register	Callee
x10-x11	a0-a1	Fn args/return values	Caller
x12-x17	a2-a7	Fn args	Caller
x18-x27	s2-s11	Saved registers	Callee
x28-x31	t3-t6	Temporaries	Caller

RV32I Base Integer Instructions – Arithmetic Operations

Inst	Name	FMT	Opcode	funct3	funct7	Description (C)	Note
add rd, rs1, rs2	ADD	R	0110011	0x0	0x00	rd = rs1 + rs2	
sub rd, rs1, rs2	SUB	R	0110011	0x0	0x20	rd = rs1 - rs2	
slt rd, rs1, rs2	Set Less Than	R	0110011	0x2	0x00	rd = (rs1 < rs2)?1:0	
sltu rd, rs1, rs2	Set Less Than (U)	R	0110011	0x3	0x00	rd = (rs1 < rs2)?1:0	zero-extends
addi rd, rs1, imm	ADD Immediate	I	0010011	0x0		rd = rs1 + imm	
slti rd, rs2, imm	Set Less Than Imm	I	0010011	0x2		rd = (rs1 < imm)?1:0	
sltiu rd, rs1, imm	Set Less Than Imm (U)	I	0010011	0x3		rd = (rs1 < imm)?1:0	zero-extends
lui rd, imm	Load Upper Imm	U	0110111			rd = imm << 12	
auipc rd, imm	Add Upper Imm to PC	U	0010111			rd = PC + (imm << 12)	

RV32I Base Integer Instructions – Logical Operations

Inst	Name	FMT	Opcod	funct3	funct7	Description (C)	Note
xor rd, rs1, rs2	XOR	R	0110011	0x4	0x00	rd = rs1 ^ rs2	
or rd, rs1, rs2	OR	R	0110011	0x6	0x00	rd = rs1 | rs2	
and rd, rs1, rs2	AND	R	0110011	0x7	0x00	rd = rs1 & rs2	
sll rd, rs1, rs2	Shift Left Logical	R	0110011	0x1	0x00	rd = rs1 << rs2	
srl rd, rs1, rs2	Shift Right Logical	R	0110011	0x5	0x00	rd = rs1 >> rs2	
sra rd, rs1, rs2	Shift Right Arith*	R	0110011	0x5	0x20	rd = rs1 >> rs2	msb-extends
xori rd, rs1, imm	XOR Immediate	I	0010011	0x4		rd = rs1 ^ imm	
ori rd, rs1, imm	OR Immediate	I	0010011	0x6		rd = rs1 | imm	
andi rd, rs1, imm	AND Immediate	I	0010011	0x7		rd = rs1 & imm	
slli rd, rs1, imm	Shift Left Logical Imm	I	0010011	0x1	imm[5:11]=0x00	rd = rs1 << imm[0:4]	
srli rd, rs1, imm	Shift Right Logical Imm	I	0010011	0x5	imm[5:11]=0x00	rd = rs1 >> imm[0:4]	
srai rd, rs1, imm	Shift Right Arith Imm	I	0010011	0x5	imm[5:11]=0x20	rd = rs1 >> imm[0:4]	msb-extends

RV32I Base Integer Instructions – Load/Store Operations

Inst	Name	FMT	Opcode	funct3	funct7	Description (C)	Note
lb rd, imm(rs1)	Load Byte	I	0000011	0x0		rd = M[rs1+imm][0:7]	
lh rd, imm(rs1)	Load Half	I	0000011	0x1		rd = M[rs1+imm][0:15]	
lw rd, imm(rs1)	Load Word	I	0000011	0x2		rd = M[rs1+imm][0:31]	
lbu rd, imm(rs1)	Load Byte (U)	I	0000011	0x4		rd = M[rs1+imm][0:7]	zero-extends
lhu rd, imm(rs1)	Load Half (U)	I	0000011	0x5		rd = M[rs1+imm][0:15]	zero-extends
sb rs2, imm(rs1)	Store Byte	S	0100011	0x0		M[rs1+imm][0:7] = rs2[0:7]	
sh rs2, imm(rs1)	Store Half	S	0100011	0x1		M[rs1+imm][0:15] = rs2[0:15]	
sw rs2, imm(rs1)	Store Word	S	0100011	0x2		M[rs1+imm][0:31] = rs2[0:31]	

RV32I Base Integer Instructions – Branching

Inst	Name	FMT	Opcode	funct3	funct7	Description (C)	Note
`beq rs1, rs2, imm`	Branch ==	B	1100011	0x0		`if(rs1 == rs2) PC += imm`	
`bne rs1, rs2, imm`	Branch !=	B	1100011	0x1		`if(rs1 != rs2) PC += imm`	
`blt rs1, rs2, imm`	Branch <	B	1100011	0x4		`if(rs1 < rs2) PC += imm`	
`bge rs1, rs2, imm`	Branch ≥	B	1100011	0x5		`if(rs1 >= rs2) PC += imm`	
`bltu rs1, rs2, imm`	Branch < (U)	B	1100011	0x6		`if(rs1 < rs2) PC += imm`	zero-extends
`bgeu rs1, rs2, imm`	Branch ≥ (U)	B	1100011	0x7		`if(rs1 >= rs2) PC += imm`	zero-extends
`jal rd, imm`	Jump And Link	J	1101111			`rd = PC+4; PC += imm`	
`jalr rd, imm(rs1)`	Jump And Link Reg	I	1100111	0x0		`rd = PC+4; PC = rs1 + imm`	

RV32I Base Integer Instructions – Misc

Inst	Name	FMT	Opcode	funct3	funct7	Description (C)	Note
`ecall`	Environment Call	I	1110011	0x0	imm=0x0	`Transfer control to OS`	
`ebreak`	Environment Break	I	1110011	0x0	imm=0x1	`Transfer control to debugger`	

RV32M Multiply Extension

Inst	Name	FMT	Opcode	funct3	funct7	Description (C)
`mul rd, rs1, rs2`	MUL	R	0110011	0x0	0x01	`rd = (rs1 * rs2)[31:0]`
`mulh rd, rs1, rs2`	MUL High	R	0110011	0x1	0x01	`rd = (rs1 * rs2)[63:32]`
`mulsu rd, rs1, rs2`	MUL High (S) (U)	R	0110011	0x2	0x01	`rd = (rs1 * rs2)[63:32]`
`mulu rd, rs1, rs2`	MUL High (U)	R	0110011	0x3	0x01	`rd = (rs1 * rs2)[63:32]`
`div rd, rs1, rs2`	DIV	R	0110011	0x4	0x01	`rd = rs1 / rs2`
`divu rd, rs1, rs2`	DIV (U)	R	0110011	0x5	0x01	`rd = rs1 / rs2`
`rem rd, rs1, rs2`	Remainder	R	0110011	0x6	0x01	`rd = rs1 % rs2`
`remu rd, rs1, rs2`	Remainder (U)	R	0110011	0x7	0x01	`rd = rs1 % rs2`

Pseudo Instructions

Pseudoinstruction	Base Instruction(s)	Meaning
`la rd, symbol`	`auipc rd, symbol[31:12]` `addi rd, rd, symbol[11:0]`	Load address
`l{b|h|w|d} rd, symbol`	`auipc rd, symbol[31:12]` `l{b|h|w|d} rd, symbol[11:0](rd)`	Load global
`s{b|h|w|d} rd, symbol, rt`	`auipc rt, symbol[31:12]` `s{b|h|w|d} rd, symbol[11:0](rt)`	Store global
`nop`	`addi x0, x0, 0`	No operation
`li rd, immediate`	*Myriad sequences*	Load immediate
`mv rd, rs`	`addi rd, rs, 0`	Copy register
`not rd, rs`	`xori rd, rs, -1`	One's complement
`neg rd, rs`	`sub rd, x0, rs`	Two's complement
`seqz rd, rs`	`sltiu rd, rs, 1`	Set if = zero
`snez rd, rs`	`sltu rd, x0, rs`	Set if != zero
`sltz rd, rs`	`slt rd, rs, x0`	Set if < zero
`sgtz rd, rs`	`slt rd, x0, rs`	Set if > zero
`beqz rs, offset`	`beq rs, x0, offset`	Branch if = zero
`bnez rs, offset`	`bne rs, x0, offset`	Branch if != zero
`blez rs, offset`	`bge x0, rs, offset`	Branch if ≤ zero
`bgez rs, offset`	`bge rs, x0, offset`	Branch if ≥ zero
`bltz rs, offset`	`blt rs, x0, offset`	Branch if < zero
`bgtz rs, offset`	`blt x0, rs, offset`	Branch if > zero
`bgt rs, rt, offset`	`blt rt, rs, offset`	Branch if >
`ble rs, rt, offset`	`bge rt, rs, offset`	Branch if ≤
`bgtu rs, rt, offset`	`bltu rt, rs, offset`	Branch if >, unsigned
`bleu rs, rt, offset`	`bgeu rt, rs, offset`	Branch if ≤, unsigned
`j offset`	`jal x0, offset`	Jump
`jal offset`	`jal x1, offset`	Jump and link
`jr rs`	`jalr x0, rs, 0`	Jump register
`jalr rs`	`jalr x1, rs, 0`	Jump and link register
`ret`	`jalr x0, x1, 0`	Return from subroutine
`call offset`	`auipc x1, offset[31:12]` `jalr x1, x1, offset[11:0]`	`Call far-away subroutine`

Ejemplos comunes

Cargar en registro el **valor** almacenado en una posición de memoria asociada a una etiqueta	`.data`
	`  var1: .word 5`
	`.text`
	`  lw t0, var1`
		Carga en **`t0`** el valor **`5`** que hay en la posición de memoria asociada a la etiqueta **`var1`**

Cargar en un registro la **dirección** de un vector	`.data`
	`  vect: .word 1, 2, 3, 4, 5`
	`.text`
	`  la t0, vect`
		Carga en **`t0`** la dirección de memoria asociada a **`vect`** para poder luego usarla para cargar los valores almacenados en dicho vector (`lw t1, n(t0)`)

Copiar el contenido de un registro en otro	`mv t0, t1`	Copia en el registro **`t0`** el valor almacenado en el registro **`t1`**
Salto incondicional a una etiqueta	`j fin`	Salta a la posición de memoria que ocupa la instrucción que hay a continuación de la etiqueta **`fin`**:

Saltos condicionales **comparando** con **cero**	`bgtz t1, fin`
	`bnez t1, fin`
	`blez t1, fin`
	…
		Comparan el valor almacenado en **`t1`** con **`0`** y en caso de cumplirse la condición (bgt - mayor, bne - distinto, ble - menor o igual, …) salta a la posición de memoria que ocupa la instrucción que hay a continuación de la etiqueta **`fin`**:

Llamada a una función "func"	`call func`	Salta a la posición de memoria que ocupa la instrucción que hay a continuación de la etiqueta **`func`**: y almacena en el registro **`ra`** la dirección de la siguiente instrucción a la instrucción **`call`**
Retorno de una función	`ret`	Salta a la posición de memoria almacenada en el registro `ra`

Salvar varios registros en la pila (**push**)	`addi sp, sp, -12`
	`sw s0, 0(sp)`
	`sw s1, 4(sp)`
	`sw s2, 8(sp)`
		Reserva 3 palabra en la pila (**`sp - 3*4`**) y luego almacena los valores almacenados en los registros **`s0`**, **`s1`** y **`s2`** en las 3 primeras palabras (+0, +4, +8) de memoria a partir de **sp**

Restaurar varios registros de la pila (**pop**)	`lw s0, 0(sp)`
	`lw s1, 4(sp)`
	`lw s2, 8(sp)`
	`addi sp, sp, 12`
		Carga los valores almacenados en las 3 primeras palabras de memoria a partir de **sp** (+0, +4, +8) en los registros **`s0`**, **`s1`** y **`s2`** y luego libera de la pila el espacio de dichas 3 palabras (**`sp + 3*4`**)

Convenio de llamadas (relajado)

Paso de parámetros a función	Los ocho primeros parámetros se pasan por los registros **`a0..a7`** El resto de parámetros se pasan por pila
Retorno de valor por una función	El valor de retorno de la función se pasa por **`a0`** (**`a0/a1`** si es de 64 bits)
Preservar valor de los registros	La función que llama (**caller**) debe preservar (salvar): • Los registros temporales (**`t0..t6`**) que no quiera perder • Los registros de parámetros (**`a0..a7`**) que no quiera perder La función llamada (**callee**) debe preservar (salvar): • Los registros salvados (**`s0..s11`**) que modifique • El registro de dirección de retorno (**`ra`**) si lo modifica • El puntero de pila (**sp**) si lo modifica
Manejo de pila	El manejo de la pila es "**full-descending**": • Para **apilar (push)**: primero se decrementa **sp** y luego se almacena (con desplazamientos 0, 4, 8, …) • Para **desapilar (pop)**: primero se carga (con desplazamientos 0, 4, 8, …) y luego se incrementa **sp** El puntero de pila **sp** siempre tiene que ser **múltiplo de 4 bytes** (se incrementa/decrementa siempre en múltiplos de 4)

Directivas del ensamblador

Directiva	Descripción
`.text`	Declara el comienzo de la sección de instrucciones
`.data`	Declara el comienzo de la sección de variables globales con valor inicial
`.bss`	Declara el comienzo de la sección de variables globales sin valor inicial
`.word` w_1, … w_n	Reserva espacio en memoria para *n* palabras inicializadas a $w_1 \dots w_n$
`.half` h_1, … h_n	Reserva espacio en memoria para *n* medias palabras inicializadas a $h_1 \dots h_n$
`.byte` b_1, … b_n	Reserva espacio en memoria para *n* bytes inicializadas a $b_1 \dots b_n$
`.zero n`	Reserva espacio en memoria para *n* bytes inicializados a 0
`.space n`	Reserva espacio en memoria para *n* bytes sin inicializar
`.string "str"`	Reserva espacio en memoria inicializado con la cadena *"str"*
`.align n`	Alinea los datos/instrucciones a direcciones múltiplo de 2^n
`.equ sym, val`	Define una constante simbólica llamada *sym* de valor *val*
`.global sym`	Hace visible la etiqueta *sym* fuera del archivo que la contiene (global)
`.extern sym`	Indica que un símbolo está definido en otro archivo del proyecto
`.end`	Declara el final del programa ensamblador

Código ejemplo

```
.data                           # comienzo del segmento de datos
    vect: .word 1, 2, 3, 4, 5   # declaración de un vector vect de 5 elementos
    len:  .word 5               # declaración de una variable len inicializada a 5
    res:  .space 4              # reserva de espacio para almacenar el resultado (res)

.global main                    # declaración de la función main como global
.text                           # comienzo del segmento de código
    main:                       # comienzo de la función principal main
      la   a0, vect             # carga de la dirección del vector vect en a0 (1er parámetro)
      lw   a1, len              # cargar el valor de la variable len en a1 (2º parámetro)
      call sumvec               # invocación de la función sumvec
      sw   a0, res, t0          # almacenar resultado de la función (a0) en res
    end:
      j    end                  # bucle infinito para terminar el código

    sumvec:                     # declaración de la función sumvec (suma elementos vector)
      addi sp, sp, -4           # se hace hueco en la pila (sp) para almacenar 1 palabra (4 bytes)
      sw   s0, 0(sp)            # se almacena s0 en la cima de la pila (s0 registro salvado)
      li   s0, 0                # se inicializa s0 a 0 (suma parcial)
      svloop:                   # cabecera del bucle que recorre el vector
          beqz a1, svexit       # salir del bucle si a1 (número elementos) es 0
          lw   t0, 0(a0)        # cargar un elemento del vector en t0 (a0 dirección del vector)
          add  s0, s0, t0       # sumar el valor leído (t0) a la suma parcial (s0)
          addi a0, a0, 4        # avanzar al siguiente elemento del vector
          addi a1, a1, -1       # decrementar contador de elementos del vector
          j    svloop           # volver al comienzo del bucle
      svexit:
      mv   a0, s0               # copiar suma parcial en a0 (valor devuelto por función)
      lw   s0, 0(sp)            # restaurar valor inicial de s0
      addi sp, sp 4             # restaurar valor inicial de sp
      ret                       # retornar de la función
```